

Batch 2 — Operators: The Workbench

Goal: understand what Reactor operators actually do to a request after Spring subscribes, and learn how to choose the right operator under interview pressure.

0. Operator Mental Model First

Before learning individual operators, fix this mental model:

- A `Mono` or `Flux` is the road.
- Operators are checkpoints placed on that road.
- Signals flow through those checkpoints only after subscription starts execution.

So if you write:

```
Mono<OrderView> pipeline = serviceA()
    .flatMap(a -> serviceB(a.id()))
    .map(b -> new OrderView(b))
    .doOnNext(view -> log.info("ready"))
    .onErrorResume(ex -> fallback());
```

then before subscription:

- no HTTP call is required to have completed yet
- no `map` has run yet
- no `doOnNext` has run yet
- no fallback has run yet

You only built the operator chain.

After Spring subscribes:

- source publishers start producing signals
- each operator reacts to those signals
- final values are serialized to JSON and written to the HTTP response

The Five Kinds of Operators You Must Recognize

Most interview questions in this batch reduce to this classification:

1. **Transform operators:** `map`
2. **Async expansion operators:** `flatMap`, `concatMap`, `switchMap`
3. **Filtering and limiting operators:** `filter`, `take`, `skip`, `distinct`
4. **Observation operators:** `doOn...`
5. **Combination and recovery operators:** `zip`, `merge`, `concat`, `combineLatest`, `onErrorResume`, `retry`

The Request-Flow Lens For Batch 2

For every operator, ask these questions:

- Does it transform one signal or create more work?
- Does it preserve order?
- Does it subscribe to inner publishers concurrently or sequentially?
- Can it cancel upstream?
- Does it alter data or only observe it?
- Does it wait for all sources or emit as soon as something arrives?

That is how senior engineers choose operators.

Decision Cheat Sheet

- Need to transform one value into another value synchronously: `map`
 - Need to call another async source per item: `flatMap`
 - Need that async call per item but in source order: `concatMap`
 - Need to cancel old work and keep only latest: `switchMap`
 - Need to drop unwanted items: `filter`
 - Need only first `n` items: `take`
 - Need to ignore first `n` items: `skip`
 - Need to deduplicate: `distinct`
 - Need logging/metrics/cleanup: `doOn...`
 - Need all independent sources together: `zip`
 - Need to consume multiple streams as they arrive: `merge`
 - Need publishers one after another: `concat`
 - Need latest value from changing sources: `combineLatest`
 - Need fallback or retry on failure: `onErrorResume`, `retryWhen`
-

6. `map` vs `flatMap` vs `concatMap` vs `switchMap`

What They Change in the Request Flow

These operators answer one question:

"When one signal arrives, what do I do next?"

- `map` says: transform this value immediately into another value.
- `flatMap` says: this value requires another async publisher.
- `concatMap` says: do async publisher work, but one item at a time in order.
- `switchMap` says: if a newer value arrives, cancel the older async work and keep only the latest.

Simple Analogy

Think of a courier desk:

- `map` = relabel the same package.
- `flatMap` = send the package to another department for more work.

- `concatMap` = send packages one by one, only after the previous one returns.
- `switchMap` = if a newer package arrives, throw away the old one and process only the newest.

The Core Difference

map

- input: one value
- output: one new value
- synchronous transformation
- no new subscription to another async source

```
Mono<UserDto> dtoMono = userMono.map(user -> new UserDto(user.id(), user.name()));
```

Use `map` when the function returns a plain object, not another `Mono` or `Flux`.

flatMap

- input: one value
- output: another publisher
- Reactor subscribes to that inner publisher
- order is not guaranteed when used on a multi-item `Flux`

```
Mono<OrderView> result = orderMono.flatMap(order -> paymentClient.getPayment(order.paymentId())
    .map(payment -> new OrderView(order, payment)));
```

Use `flatMap` when your next step is another async call.

concatMap

- like `flatMap`, but preserves source order
- subscribes to one inner publisher at a time
- useful when order matters more than maximum concurrency

```
Flux<OrderDetails> details = orderIds.concatMap(orderService::getOrderDetails);
```

If `orderIds` are 1, 2, 3, then `concatMap` makes sure result order stays 1, 2, 3 even if 2 could finish faster.

switchMap

- when a new outer value arrives, old inner work is cancelled
- best for typeahead, live search, latest-state dashboards

```
Flux<SearchResult> results = queryFlux.switchMap(searchClient::search);
```

If the user types `m`, then `ma`, then `mar`, old searches are cancelled and only the latest matters.

Why This Matters in Request Flow

Suppose request flow is:

1. fetch customer
2. fetch payment using customer data
3. build response

Then:

- use `flatMap` between step 1 and step 2 because step 2 is another async call
- use `map` at the end because building response DTO is plain synchronous object creation

When Each One Wins

- `map` : DTO transformation, enrichment from already available data
- `flatMap` : service call, DB call, cache lookup, async file/network step
- `concatMap` : preserve order for stream processing or rate-limited downstream
- `switchMap` : latest-only semantics, cancel stale work

Backfires That Might Occur

- Using `map` when the lambda returns `Mono<T>` gives nested publishers like `Mono<Mono<T>>`
- Using `flatMap` on `Flux` and expecting order to stay unchanged
- Using `concatMap` where concurrency is required and accidentally slowing the pipeline
- Using `switchMap` in workflows where dropping previous work is not acceptable

Anti-Patterns

- `map(id -> service.get(id))` when `service.get(id)` returns `Mono<T>`
- `flatMap` everywhere without understanding concurrency or order
- `switchMap` for payment or write flows where cancellation loses business work
- `concatMap` for independent slow calls when parallel fan-out would be better

Code Sample (Java)

```
Mono<CustomerSummary> summary = customerClient.getCustomer(customerId)
    .flatMap(customer -> loyaltyClient.getPoints(customer.id())
        .map(points -> new CustomerSummary(customer, points)));

Flux<OrderDetails> ordered = Flux.just("o1", "o2", "o3")
    .concatMap(orderService::getOrderDetails);

Flux<SearchResult> latestOnly = queryFlux.switchMap(searchClient::search);
```

Interview Trap

"`flatMap` is just async `map`, so it is always the right choice for modern code."

That is wrong. `flatMap` changes concurrency and ordering behavior. Use it only when the next step returns another publisher.

Quick Revision Notes

- `map` = sync value-to-value transformation

- `flatMap` = async publisher expansion
 - `concatMap` = async expansion with order preserved
 - `switchMap` = cancel stale inner work, keep latest
 - Trap to remember = `flatMap` on `Flux` can interleave results
-

7. `filter`, `take`, `skip`, `distinct`

What They Change in the Request Flow

These operators decide which signals are allowed to keep moving.

- `filter` removes items that fail a predicate
- `take` stops after enough items arrive
- `skip` ignores the first items
- `distinct` removes duplicates

These are not just data-shaping tools. Some of them affect cancellation and upstream demand.

Simple Analogy

Think of airport security:

- `filter` = allow only passengers with valid documents
- `take(5)` = after first 5 passengers are admitted, close the gate
- `skip(2)` = ignore first 2 passengers
- `distinct` = do not let duplicate tickets through twice

Operator-by-Operator

`filter`

```
Flux<Order> paidOrders = orders.filter(order -> order.status() == Status.PAID);
```

Only matching items move downstream.

`take`

```
Flux<Event> topFive = eventFlux.take(5);
```

After 5 items, Reactor can cancel upstream because it has enough data.

This is important. `take` is not just a list trim at the end. It can stop further production.

`skip`

```
Flux<Event> afterWarmup = eventFlux.skip(10);
```

Useful when initial items are noise, warmup, or already processed.

distinct

```
Flux<Order> uniqueOrders = orderFlux.distinct(Order::id);
```

Deduplicates across the whole stream based on a key.

Also know the interview distinction:

- `distinct` = deduplicate globally across seen items
- `distinctUntilChanged` = drop only consecutive duplicates

Why This Matters in Request Flow

Suppose a recommendation service returns a noisy `Flux<Recommendation>`.

You may want:

```
recommendationClient.getRecommendations(customerId)
    .filter(Recommendation::isEligible)
    .distinct(Recommendation::id)
    .take(3);
```

This means:

- ignore ineligible recommendations
- remove duplicates
- stop after 3 useful results

That can reduce unnecessary downstream processing and even cancel the upstream stream early.

Backfires That Might Occur

- `take` unexpectedly cancels upstream and surprises teams monitoring connection behavior
- `distinct` on huge or infinite streams can consume memory because it must remember seen keys
- `skip` can hide important initial events if used carelessly
- `filter` can produce an empty stream and cause later assumptions to fail

Anti-Patterns

- Using `take` and assuming upstream still keeps producing normally for this subscriber
- Using `distinct` blindly on unbounded streams
- Forgetting to handle the case where `filter` removes everything
- Treating these operators as if they only modify final collections after the fact

Code Sample (Java)

```
Flux<ProductCard> cards = recommendationClient.getProducts(customerId)
    .filter(ProductCard::inStock)
    .distinct(ProductCard::id)
    .take(5)
    .skip(1);
```

Interview Trap

"`take(5)` just means collect the first 5 after the source finishes."

That is wrong. `take(5)` can trigger early cancellation after the fifth item.

Quick Revision Notes

- `filter` = keep matching items only
 - `take(n)` = stop after `n`, can cancel upstream
 - `skip(n)` = ignore first `n`
 - `distinct` = global dedupe, memory cost matters
 - Trap to remember = `take` affects lifecycle, not just the final output shape
-

8. `doOn...` Callbacks (Lifecycle Hooks)

What They Change in the Request Flow

Strictly speaking, these operators should not change business flow.

They are there to observe signals passing through the pipeline.

That is why they are perfect for:

- logging
- metrics
- tracing
- cleanup
- debugging

But they are dangerous if you treat them as business logic operators.

Simple Analogy

Think of CCTV cameras in a warehouse.

- Cameras observe packages moving through stations.
- Cameras do not change the package contents.
- If you try to run the business through the cameras, the design is broken.

Most Important Hooks

- `doOnSubscribe` = subscription started
- `doOnRequest` = downstream demanded data
- `doOnNext` = item passed through
- `doOnError` = error passed through
- `doOnComplete` = completion for `Flux`
- `doOnSuccess` = success for `Mono`
- `doOnCancel` = downstream cancelled
- `doFinally` = always runs at end with completion, error, or cancel signal

Why This Matters in Request Flow

Suppose request pipeline is:

```
return paymentClient.getPayment(orderId)
    .doOnSubscribe(sub -> log.info("payment fetch started"))
    .doOnNext(payment -> log.info("payment received"))
    .doOnError(ex -> log.error("payment failed", ex))
    .doFinally(signal -> metrics.recordCompletion(signal));
```

These hooks help you answer:

- did Spring subscribe?
- did the upstream call actually return anything?
- did the request fail or get cancelled?
- did cleanup happen even on failure?

What `doOn...` Does Not Do

It should not be the main place for:

- DTO building
- mutating domain state
- calling another service
- persistence writes that the flow depends on

Why?

Because `doOn...` is for side effects around the signal, not the main transformation contract.

Backfires That Might Occur

- duplicate side effects if the publisher is subscribed multiple times
- hidden business logic inside logging hooks
- confusion about why side effects did not run when there was no subscription
- cleanup not happening if the wrong hook is chosen instead of `doFinally`

Anti-Patterns

- saving to DB inside `doOnNext` as core behavior
- putting critical logic in `doOnError` without changing the main flow
- expecting `doOnNext` to run if upstream completes empty
- using hooks as a substitute for proper operators like `map`, `flatMap`, or `onErrorResume`

Code Sample (Java)

```
Mono<Invoice> invoiceMono = invoiceClient.getInvoice(invoiceId)
    .doOnSubscribe(sub -> log.info("invoice fetch started"))
    .doOnNext(invoice -> log.info("invoice fetched: {}", invoice.id()))
    .doOnError(ex -> log.error("invoice fetch failed", ex))
    .doOnCancel(() -> log.warn("invoice fetch cancelled"))
    .doFinally(signal -> log.info("invoice flow ended with {}", signal));
```

Interview Trap

"doOnNext is a good place for the real business transformation because it sees every item."

That is wrong. Use `map` or `flatMap` for business transformations. Use `doOn...` for observation and side effects.

Quick Revision Notes

- `doOn...` hooks observe signal flow
- they run only when subscription and signal movement happen
- `doFinally` is the safest all-path cleanup hook
- Trap to remember = `doOn...` is not your main business pipeline

9. zip, merge, concat, combineLatest

What They Change in the Request Flow

These operators answer another important question:

"I have multiple publishers. How should their results be coordinated?"

- `zip` = wait for one value from each and combine them together
- `merge` = subscribe to all and pass values downstream as soon as they arrive
- `concat` = subscribe to the next publisher only after the previous completes
- `combineLatest` = after each source has emitted once, use the latest value from each whenever any source updates

Simple Analogy

Think of delivery coordination:

- `zip` = wait until all three parcels arrive before packing the shipment
- `merge` = unload parcels onto the belt as soon as each truck arrives
- `concat` = unload truck A fully, then truck B, then truck C
- `combineLatest` = update the display board whenever any department reports new numbers, always using the latest from the others

Operator-by-Operator

zip

```
Mono<AccountView> accountView = Mono.zip(profileMono, paymentMono, loyaltyMono)
    .map(tuple -> new AccountView(tuple.getT1(), tuple.getT2(), tuple.getT3()));
```

Use `zip` when all inputs are required for the final output.

Important truth: overall latency is usually dominated by the slowest required source.

merge

```
Flux<Event> allEvents = Flux.merge(orderEvents, paymentEvents, shipmentEvents);
```

Use `merge` when values should flow as soon as they arrive, and order between sources does not matter.

concat

```
Flux<Event> sequential = Flux.concat(orderEvents, paymentEvents, shipmentEvents);
```

Use `concat` when publisher order matters and later sources must wait.

combineLatest

```
Flux<DashboardView> dashboard = Flux.combineLatest(cpuFlux, memoryFlux, trafficFlux,
    (cpu, memory, traffic) -> new DashboardView(cpu, memory, traffic));
```

Use it for live dashboards, changing sensor feeds, or UI state streams.

How To Choose Between Them

- Building one response from independent Monos: `zip`
- Merging event streams: `merge`
- Strict source order: `concat`
- Continuously updating latest state: `combineLatest`

Why This Matters in Request Flow

Suppose request needs:

- customer from service A
- payment from service B
- loyalty from DB

If all three are required for one final JSON response, `zip` is the correct fit.

Why not `merge`?

Because `merge` emits as soon as values arrive, but your final response needs all three pieces.

Why not `concat` ?

Because those calls are independent and do not need sequential waiting.

Why not `combineLatest` ?

Because this is not a continuously updating dashboard. It is one request requiring one aggregate response.

Backfires That Might Occur

- using `merge` and then being surprised by interleaving order
- using `concat` for independent calls and paying avoidable latency
- using `zip` when one source may never emit, causing the combination to wait forever
- using `combineLatest` when only one final aggregate is needed

Anti-Patterns

- using `zip` for streams where not every source reliably emits
- using `merge` when ordered response assembly is needed
- using `concat` where parallel fan-out should happen
- confusing `combineLatest` with `zip`

Code Sample (Java)

```
Mono<CustomerDashboard> dashboard = Mono.zip(
    customerClient.getCustomer(customerId),
    paymentClient.getPayment(customerId),
    loyaltyRepository.findById(customerId)
)
.map(tuple -> new CustomerDashboard(tuple.getT1(), tuple.getT2(), tuple.getT3()));
```

Interview Trap

"`zip` is just parallel `merge`."

That is wrong. `zip` coordinates one item from each source and emits combined tuples. `merge` emits items independently as they arrive.

Quick Revision Notes

- `zip` = all required together
 - `merge` = emit as soon as ready
 - `concat` = one publisher after another
 - `combineLatest` = latest-state recomputation
 - Trap to remember = `zip` waits for all required sources and is gated by the slowest one
-

10. Error Handling Operators

What They Change in the Request Flow

These operators decide what the pipeline does when an error signal appears.

That matters because errors in Reactor are signals, not just thrown exceptions flying around uncontrolled.

Key rule:

- an error is terminal for that source unless you recover with another publisher

Simple Analogy

Think of a highway route:

- `onErrorReturn` = use a fixed detour destination
- `onErrorResume` = calculate a new route dynamically
- `onErrorMap` = relabel the incident into a better business exception
- `retry` = try the same road again
- `retryWhen` = retry with policy, delay, or backoff

Operator-by-Operator

`onErrorReturn`

```
Mono<Price> priceMono = priceClient.getPrice(productId)
    .onErrorReturn(Price.unavailable());
```

Use for simple fixed fallback values.

`onErrorResume`

```
Mono<Price> priceMono = priceClient.getPrice(productId)
    .onErrorResume(TimeoutException.class, ex -> cacheService.getCachedPrice(productId));
```

Use when fallback depends on the exception or needs another reactive call.

`onErrorMap`

```
Mono<Payment> paymentMono = paymentClient.getPayment(paymentId)
    .onErrorMap(ex -> new PaymentServiceException("Payment fetch failed", ex));
```

Use to convert technical errors into domain or API-specific errors.

`retry`

```
Mono<Payment> paymentMono = paymentClient.getPayment(paymentId)
    .retry(2);
```

Simple retry, but dangerous if the operation has side effects.

retryWhen

```
Mono<Payment> paymentMono = paymentClient.getPayment(paymentId)
    .retryWhen(Retry.backoff(3, Duration.ofMillis(200)));
```

Use for controlled retries with policy.

Scope Rule That Interviewers Love

Error handlers catch errors from upstream before their position in the chain.

Example:

```
source()
    .map(this::step1)
    .onErrorResume(ex -> fallback())
    .map(this::step2);
```

This `onErrorResume` handles:

- errors from `source()`
- errors from `step1`

It does **not** handle errors thrown later in `step2`.

That scope rule matters a lot.

Why This Matters in Request Flow

Suppose payment service times out, but you have cached payment status.

Then request flow can be:

1. start payment call
2. timeout or remote failure occurs
3. error signal travels downstream
4. `onErrorResume` catches it
5. fallback cache publisher is subscribed
6. pipeline continues successfully

This is still reactive execution. You are not leaving the chain. You are switching to another publisher.

Backfires That Might Occur

- using `retry` on non-idempotent POST operations and duplicating side effects
- swallowing important errors with overly broad `onErrorResume`
- putting error operator in the wrong location and missing downstream failures
- using fixed fallback values that hide systemic outages

Anti-Patterns

- `onErrorResume(ex -> Mono.empty())` everywhere without business justification
- `retrying` writes blindly
- converting every error into success and losing observability
- misunderstanding that `retry` means resubscription to the source

Code Sample (Java)

```
Mono<PaymentStatus> paymentStatus = paymentClient.getPayment(customerId)
    .timeout(Duration.ofSeconds(2))
    .onErrorResume(TimeoutException.class, ex -> paymentCache.getStatus(customerId))
    .onErrorMap(ex -> new PaymentFlowException("Could not resolve payment status", ex));
```

Interview Trap

"`retry` just continues from where the previous failure stopped."

That is wrong. `retry` means resubscribe to the source. The upstream work starts again.

Quick Revision Notes

- errors are signals in Reactor
- `onErrorReturn` = fixed fallback
- `onErrorResume` = alternate publisher fallback
- `onErrorMap` = transform exception type
- `retry` / `retryWhen` = resubscribe, so idempotency matters

Batch 2 — Putting Operators Into One Request Story

Now connect everything using one realistic request.

Use Case

Endpoint: `GET /customers/{id}/dashboard`

Need to build response from:

- customer profile service
- payment service
- recommendation service
- loyalty data from reactive DB

Business rules:

- customer must be active
- recommendations should be unique and only top 3 eligible ones
- payment timeout should fall back to cache
- final response should include all required pieces
- log lifecycle clearly

The Code

```
@GetMapping("/customers/{id}/dashboard")
Mono<CustomerDashboard> dashboard(@PathVariable String id) {
    Mono<Customer> customerMono = customerClient.getCustomer(id)
        .filter(Customer::active)
        .switchIfEmpty(Mono.error(new IllegalStateException("Inactive or missing customer")));

    Mono<PaymentStatus> paymentMono = paymentClient.getPayment(id)
        .timeout(Duration.ofSeconds(2))
        .onErrorResume(TimeoutException.class, ex -> paymentCache.getStatus(id));

    Mono<Loyalty> loyaltyMono = loyaltyRepository.findByCustomerId(id);

    Mono<List<Recommendation>> recommendationsMono = recommendationClient.getRecommendations(id)
        .filter(Recommendation::eligible)
        .distinct(Recommendation::id)
        .take(3)
        .collectList();

    return customerMono
        .flatMap(customer -> Mono.zip(paymentMono, loyaltyMono, recommendationsMono)
            .map(tuple -> new CustomerDashboard(
                customer,
                tuple.getT1(),
                tuple.getT2(),
                tuple.getT3()
            )))
        .doOnSubscribe(sub -> log.info("dashboard request started for {}", id))
        .doOnSuccess(result -> log.info("dashboard request succeeded for {}", id))
        .doOnError(ex -> log.error("dashboard request failed for {}", id, ex))
        .doFinally(signal -> metrics.recordDashboardCompletion(id, signal));
}
```

Why Each Operator Was Chosen

- `filter` = allow only active customer
- `flatMap` = once customer arrives, continue with async dependent work
- `zip` = payment, loyalty, and recommendations are all needed for final response
- `map` = build final DTO synchronously
- `distinct` = remove duplicate recommendations
- `take(3)` = stop after enough recommendations, can cancel upstream early
- `doOn...` = observe lifecycle and record logs/metrics
- `onErrorResume` = fall back from payment timeout to cache

Why Not Other Operators?

- not `map` instead of `flatMap`, because the next step is async and returns a publisher
- not `merge`, because one final dashboard requires all pieces together
- not `concat`, because payment, loyalty, and recommendations are independent and can happen in parallel

- not `switchMap`, because a normal request does not mean "cancel previous and keep latest"

Full Request Life Journey

Phase 1: Request Hits WebFlux

1. Browser calls `GET /customers/42/dashboard`
2. Netty receives request on an event loop thread
3. Spring routes request to this controller

Phase 2: Assembly Happens

Controller method runs and builds:

- `customerMono`
- `paymentMono`
- `loyaltyMono`
- `recommendationsMono`
- final pipeline with `flatMap`, `zip`, `map`, `doOn...`

At this point, you built the recipe.

Usually, real execution has not started yet.

Phase 3: Spring Subscribes

After controller returns the final `Mono<CustomerDashboard>`, Spring subscribes because it needs real data to write the HTTP response.

Now execution starts.

Phase 4: Customer Signal Arrives First

- profile service responds with customer
- `filter(Customer::active)` checks whether customer is active
- if inactive, the stream becomes empty and `switchIfEmpty` converts that into an error
- if active, `flatMap` continues to the next async stage

Phase 5: Parallel Fan-Out Inside `zip`

Now these start for the same request:

- payment service call
- loyalty DB query
- recommendation service stream

Suppose timing is:

- loyalty DB = 200 ms
- recommendations = emits 10 items over 800 ms
- payment service = 5 seconds or timeout at 2 seconds

Phase 6: Recommendation Stream Is Controlled

- recommendations start arriving one by one
- `filter` removes ineligible ones
- `distinct` removes duplicates
- `take(3)` stops after three useful items
- upstream recommendation stream may be cancelled early once enough items are collected
- `collectList()` turns the selected `Flux<Recommendation>` into `Mono<List<Recommendation>>`

This is a great example of operators changing not just output, but actual upstream lifecycle.

Phase 7: Payment Error Handling Happens

If payment service times out after 2 seconds:

- `timeout(...)` emits an error signal
- `onErrorResume(...)` catches that timeout
- cache fallback publisher is subscribed
- cached payment status is emitted instead

So the request survives even though the original payment call failed.

Phase 8: `zip` Waits For All Required Pieces

`zip` now coordinates:

- payment result or fallback
- loyalty DB result
- top 3 recommendations list

Only when all three are available does `zip` emit one tuple downstream.

This is the point where many candidates confuse `zip` with `merge`.

`zip` does not pass partial results just because one source finished early.

Phase 9: Final DTO Is Built

- `map(...)` runs synchronously
- `CustomerDashboard` object is created
- `doOnSuccess` observes the successful result

Phase 10: Spring Writes JSON

- Spring receives emitted `CustomerDashboard`
- Jackson serializes it
- HTTP response body is written as JSON bytes
- `doFinally` runs for cleanup/metrics

What If Operators Were Chosen Wrong?

- if you used `map` instead of `flatMap`, you would build nested publishers and the chain shape would be wrong
- if you used `merge` instead of `zip`, you would get independent arrivals instead of one final coordinated aggregate
- if you forgot `take(3)`, recommendation stream might keep running longer than necessary

- if you used `retry` blindly on a write flow, you could duplicate side effects
- if you put business logic in `doOnNext`, your pipeline would become fragile and misleading

Final Batch 2 Memory Model

- Batch 1 taught you when execution starts
- Batch 2 teaches you what happens after execution starts

Use this sentence in interviews:

"In WebFlux, operators are the control points that transform, coordinate, filter, observe, and recover signal flow after Spring subscribes to the publisher."

Batch 2 — Interview Hot Questions

1. When should I use `map` instead of `flatMap` ?

Use `map` when the lambda returns a plain value. Use `flatMap` when it returns another `Mono` or `Flux`.

2. Why does `flatMap` sometimes reorder results?

Because multiple inner publishers may run concurrently and emit as they complete.

3. Why is `concatMap` slower than `flatMap` ?

Because it preserves order by waiting for each inner publisher to complete before subscribing to the next.

4. Why is `switchMap` dangerous for business writes?

Because it cancels previous in-flight inner work when a newer outer item arrives.

5. What is the biggest hidden effect of `take(n)` ?

It can cancel upstream after enough items arrive.

6. Why is `doOnNext` not meant for core business logic?

Because it is an observation hook, not the main transformation contract.

7. When is `zip` the right choice?

When one final result depends on values from multiple independent publishers.

8. What is the biggest retry trap?

Retry means resubscription, so non-idempotent operations can run multiple times.

Batch 2 — Revision Notes

- One-line summary: operators decide how signals are transformed, filtered, coordinated, observed, and recovered after subscription begins execution.
- Three keywords: transformation, coordination, recovery.
- One trap: choosing `flatMap` or `zip` without thinking about order, concurrency, or lifecycle.
- One memory trick: `map` reshapes one box, `flatMap` sends it to another department, `zip` waits for all boxes, `take` closes the gate early.